DAILY WORK SUMMARY

SLT WhatsApp Agent Dashboard

Chat Lock Visibility Feature — Design, Implementation & Testing

Project	SLT WhatsApp Agent (Laravel) – SLBFE Chat Dashboard
Module Worked On	ChatController, ChatApiController – Chat Locking & Visibility Logic
Date	August 28, 2026
Environment	Local development (127.0.0.1:8000), tested with two agent accounts (Kosala, Charith)

1. Objective of Today's Work

The goal for today was to review the existing chat-locking behavior of the SLT WhatsApp Agent dashboard and design/implement a rule so that when one agent takes (locks) a chat, that chat is no longer visible to other agents in their chat list — preventing duplicate replies and confusion during multi-admin usage.

2. Summary of Work Completed

Task	Description	Status
System analysis	Reviewed the project setup manual and README to understand the existing chat lock mechanism, controllers, and data flow.	**Completed**
Root cause identification	Confirmed that locked chats were still visible to all agents in the sidebar and via direct URL, because no query filter excluded locked contacts.	**Completed**
Query-level fix	Updated `ChatController::contactsQuery()` to hide contacts locked by another agent, while still showing them to the agent who holds the lock.	**Completed**
Stale lock cleanup	Added logic to auto-release locks older than the configured TTL (30 minutes) before building the chat list, so chats do not stay hidden forever.	**Completed**
Live testing (2 agents)	Verified in the browser using two accounts (Kosala & Charith) that once Charith locks a chat, it disappears from Kosala's sidebar list in real time.	**Completed**
Direct-URL access gap	Identified that a locked chat can still be opened by typing its URL directly (bypasses the list filter). Fix drafted for `ChatController::show()`.	**Pending decision**

3. Key Findings

- The system has two separate "ownership" concepts that look similar but behave differently: **Human Handoff Assignment** (informational – "Assigned to X") and the **Reply Lock** (functional – `locked_by_user_id`). Only the latter should drive visibility.
- The reply lock is currently only acquired inside `ChatApiController::send()`, meaning a chat is not protected until the first reply is actually sent — simply opening a chat does not lock it.
- Hiding a chat from the sidebar list does **not** prevent direct access via URL (e.g. `/chats/2`), since `show()` loads the contact independently of the filtered list query.
- Live two-browser testing confirmed the sidebar-level fix works exactly as intended: the locked chat count and list update correctly per agent.

4. Pseudocode — Logic / Algorithm Explained Step-by-Step

4.1 Overall Rule (Plain English)

> "Show a chat to an agent UNLESS it is currently locked by a different agent. If the lock has expired (older than 30 minutes), treat it as unlocked and show it to everyone again."

4.2 Step-by-Step Breakdown

1. When any screen needs the chat list (sidebar, chat page, or auto-refresh), first sweep the database for expired locks and clear them.
2. Build the chat query with a condition: include a chat if it has no lock, OR its lock belongs to the current logged-in agent.
3. Sort the remaining (visible) chats by priority, unread count, and recency — unchanged from existing behavior.
4. Render the filtered list to the agent. A chat locked by someone else simply never appears in the array sent to the view.
5. When an agent opens a specific chat directly by URL, re-check the lock ownership before displaying it, and block access if it is locked by someone else and not expired.
6. When the lock owner releases the chat (manually, or the 30 minute TTL expires), the next list refresh automatically includes it again for all agents — no extra "unhide" step is needed because visibility is computed live from the lock fields.

4.3 Pseudocode

```
FUNCTION getVisibleChats(currentAgent):

    // Step 1: Clean up expired locks first
    FOR EACH contact IN AllContacts WHERE locked_by IS NOT NULL:
        IF (currentTime - contact.locked_at) > LOCK_TTL_SECONDS (1800):
            contact.locked_by = NULL
            contact.locked_at = NULL
            SAVE contact

    // Step 2: Filter chats -> hide if locked by someone else
    visibleChats = []
    FOR EACH contact IN AllContacts:
        isUnlocked      = (contact.locked_by IS NULL)
        isLockedByMe    = (contact.locked_by == currentAgent.id)

        IF isUnlocked OR isLockedByMe:
            ADD contact TO visibleChats
        // ELSE: locked by another agent -> skip, do not show

    // Step 3: Sort as usual (priority, unread, recency)
    SORT visibleChats BY handoffStatus, unreadCount DESC, lastMessageAt DESC
```

```
    RETURN visibleChats


FUNCTION openChatDirectly(contact, currentAgent):

    // Step 5: Guard against direct URL access bypassing the list filter
    lockedBySomeoneElse = (contact.locked_by IS NOT NULL)
                          AND (contact.locked_by != currentAgent.id)
                          AND NOT isLockExpired(contact)

    IF lockedBySomeoneElse:
        RETURN ERROR "423 Locked - this chat belongs to another agent"
    ELSE:
        RETURN renderChatView(contact)


FUNCTION sendReply(contact, currentAgent, messageText):

    LOCK DATABASE ROW FOR contact         // prevent race condition

    IF contact.locked_by IS NOT NULL AND isLockExpired(contact):
        contact.locked_by = NULL          // stale lock cleared

    IF contact.locked_by IS NOT NULL AND contact.locked_by != currentAgent.id:
        RETURN ERROR "Chat is locked by another agent, please wait"

    // Chat is free, or already locked by me -> (re)claim the lock
    contact.locked_by  = currentAgent.id
    contact.locked_at  = currentTime
    SAVE contact

    RELEASE DATABASE LOCK

    result = CallExternalWhatsAppAPI.sendMessage(contact.mobile, messageText)
    STORE result LOCALLY as outgoing message
    RETURN success
```

4.4 Why Each Step Matters

Step	Purpose
Expired-lock cleanup	Prevents a chat from being permanently hidden if an agent closes their browser or forgets to release the lock.
Unlocked OR locked-by-me filter	Ensures the lock owner never loses sight of their own active conversation while it is hidden from everyone else.
Row-level DB lock during send	Stops a race condition where two agents click "Send" at almost the same instant and both believe they own the chat.
Direct-access guard	Closes the gap where an agent could bypass the hidden sidebar list by typing the chat URL directly or using a bookmark/browser history.

5. Test Results (Live Verification)

- **Setup:** Two browser sessions, logged in as *Kosala* and *Charith*, both viewing contact `94767379205`.
- **Action:** Charith clicked "Take Chat" to acquire the lock.
- **Result on Charith's screen:** Button changed to "Chat Locked by You" (green); reply box remained active.
- **Result on Kosala's screen:** Button changed to "Locked" (red); warning banner shown: *"This chat is locked by Charith. Wait for them to release it or the lock will expire in 30 minutes."*; reply box disabled.

- **Sidebar check:** Kosala's chat list count dropped from 4 to 3 "Human chats," and the locked contact no longer appeared in his list — confirming the filter logic works as designed.
- **Outstanding gap:** Kosala could still view the full conversation by navigating directly to `/chats/2`, since that route does not yet apply the same lock check as the list query.

6. Open Decision for Tomorrow

> Decide whether a locked chat should be:
> (a) fully blocked from viewing for other agents (show a "locked" error page), or
> (b) viewable but not repliable (current behavior).
> This determines whether the guard is added to `ChatController::show()` as a hard block or left as a soft, reply-only restriction.

7. Next Steps

1. Get product decision on the view-vs-block question above.
2. If full blocking is required, add the lock check to `show()` and design a simple "This chat is locked" error page.
3. Consider whether "Take Chat" should acquire the lock immediately on open, rather than only at send time, so hiding starts the moment an agent takes a chat.
4. Add automated tests covering: lock acquisition, expiry, visibility filtering, and direct-URL access.

End of daily work summary — SLT WhatsApp Agent Dashboard project.